

C++ INTERVIEW PREP

Հիշողության մոդելը C++-ում Pointers · Stack · Heap

Ինչ է իրականում կատարվում CPU-ի և RAM-ի միջև, երբ աշխատում ես pointer-ների ու dynamic memory-ի հետ:

01 Pointer vs Reference

02 Dereference

03 Stack vs Heap

04 new / delete

05 Dangling Pointer

06 Stack Frame

Հարցազրույցի մակարդակի կոնսպեկտ

Pointer vs Reference

Pointer

address է պահում

```
int x = 10;
int* ptr = &x;    // x-ի հասցեն
*ptr = 50;         // x == 50
```

- Կարող է լինել `nullptr`
- Կարող է redirect լինել այլ object-ի
- Ունի իր memory-ն՝ `sizeof = 8B`

Reference

alias է

```
int x = 10;
int& ref = x;      // x-ի ուրիշ անունը
ref = 100;         // x == 100
```

- Պարտադիր initialize ստեղծման պահին
- Չի կարող լինել `nullptr`
- Չի կարող refer անել այլ object-ի

ԿԱՐՃ ՊԱՏԱՍԽԱՆ

Pointer-ը հասցե պահող variable է՝ կարող է `nullptr` լինել և redirect արվել: Reference-ը գոյություն ունեցող object-ի alias է՝ պարտադիր initialized, ոչ `nullptr`, ոչ rebindable:

Reference-ը չի փոխում target-ը

```
int a = 10;
int b = 20;
int& ref = a;
```

```
ref = b;    // ի՞նչ է կատարվում
```

```
cout << a;
```

Շատերը սխալ են մտածում

«ref-ը հիմա b-ին է նայում» — **ՍԽԱԼ**

Իրականում կատարվում է

```
a = b;    // a==20, b==20
```

ref դեռ a-ի alias-ն է: Output: **20**

ԻՆՉՈՒ

Reference-ին վերագրելը միշտ գրում է հենց այն object-ի մեջ, որին alias է անում: Rebind գոյություն չունի:

Dereference — ***ptr** ինչ է անում

Address	Value	
0x1000	10	<- x
0x2000	0x1000	<- ptr

1 Կարդա **ptr** = **0x1000**

2 CPU-ն գնում է **0x1000** հասցե

3 Կարդում է այնտեղի արժեքը → **10**

Assembly մակարդակում

```
; int y = *ptr;
mov rax, [ptr]    ; rax = հասցե
mov ebx, [rax]    ; ebx = տվյալը
```

Dereference = ընդամենը մեկ լրացուցիչ memory lookup.

Pointer → Address
Address → Data

ԵՐԿՈՒ ՏԱՐԲԵՐ *

int* ptr → declaration (ptr-ը pointer type է). ***ptr** → operator (գնա հասցեով, կարդա/գրի):

Stack vs Heap

Stack ավտոմատ · արագ · փոքր

- Local variables, parameters
- Return address, saved registers
- LIFO — Last In, First Out
- Allocation = `sub rsp, 4`
- Չափը սահմանափակ է (1–8 MB)

```
MyClass obj;    // dtor ավտոմատ
```

Heap ճկուն · դանդաղ · մեծ

- Runtime allocation՝ `new`
- Allocator-ը փնտրում է ազատ block
- Պետք է ձեռքով ազատել՝ `delete`
- Մոռանալը → **Memory Leak**

```
MyClass* o = new MyClass();
delete o;    // պարտադիր
```

MISCONCEPTION

Stack-ն ավելի արագ չէ, որ «CPU-ին մոտ է»: Երկուսն էլ RAM-ում են: Պատճառն allocator-ի պարզությունն է (+ լավ cache locality):

Որտե՞ղ է ապրում ամեն ինչ

```
int x = 5;
int* p = new int(10);
```

STACK

x = 5

p = 0x5000

HEAP

0x5000 ← → 10

Pointer-ը stack-ում է, object-ը heap-ում:

new-ի երեք տարբերակ

```
new int;      // ??? → Undefined Behavior
new int();    // 0
new int(42);  // 42
```

INTERVIEW

`int x;` ավելի արագ է քան `new int` — stack allocation է, allocator չի կանչվում:

delete և Dangling Pointer

```
int* p = new int(5);
delete p;
cout << *p;    // ⚠ UB
```

delete-ը pointer-ը չի ջնջում — ջնջում է object-ը, որին pointer-ը ցույց էր տալիս:

```
p = 0x5000    // դեռ հասցե ունի
0x5000 → freed // valid object չկա
```

Ինչու է սա նույնպես UB

```
int x = 10;
int* p = &x;
delete p;    // ⚠
```

delete կարելի է կանչել միայն **new**-ից ստացված հիշողության վրա: x-ը stack-ում է → delete անել չի կարելի:

UNDEFINED BEHAVIOR ՆՇԱՆԱԿՈՒՄ Է

Կարող է 5 տպել, կարող է աղբ տպել, կարող է crash / Segmentation Fault լինել — երաշխավորված վարք չկա:

Stack Frame & Return Address

```
void bar(){ int z=30; }
void foo(){ int y=20; bar(); }
int main(){ int x=10; foo(); }
```

```
bar frame    z = 30    ◀ top
foo frame    y = 20
main frame   x = 10
```

Frame-ի ներսում

- Local variables
- Parameters
- **Return Address** — ուր վերադառնա execution-ը
- Saved registers (`push rbx` / `pop rbx`)

ՈՒՇԱԴՐՈՒԹՅՈՒՆ

Return Address $\neq$ `return 5;` . Դա հաջորդ instruction-ի հասցեն է, ոչ թե function-ի արժեքը:

Function-ից հետո հիշողությունը զրոյացվում է

```
void foo(){  
    int x = 10;  
} // x-ի հետ ի՞նչ է լինում
```

Ոչ, չի զրոյացվում:

RAM-ում կարող է դեռ մնալ **10**, բայց այդ տարածքն արդեն համարվում է ազատ և կվերագրվի հաջորդ call-երով:

Stack Object

```
MyClass obj; // dtor ավտոմատ
```

Heap Object

```
MyClass* o = new MyClass();  
// delete չկա → dtor չի կանչվի → leak
```

Մեկ նախադասությամբ

Pointer

Հիշողության հասցե պահող փոփոխական:

Dereference (*p)

Գնա այդ հասցեով, կարդա/գրի տվյալը:

Stack

Ավտոմատ, արագ, փոքր — frames, locals, parameters:

Heap

Runtime allocation՝ `new` / `delete`, ճկուն բայց թանկ:

Return Address

Function-ից հետո execution-ը շարունակելու instruction-ի հասցեն:

Dangling Pointer

`delete`-ից հետո դեռ հին հասցեն պահող pointer → UB:

GOLDEN RULE

Ամեն `new`-ի դիմաց՝ մեկ `delete`. Ժամանակակից C++-ում՝ smart pointers (`unique_ptr`, `shared_ptr`):